ERPChat System Knowledge Base: Business Logic & SQL Rules

Context-Preserving Rule Definitions and Deterministic Query Logic Patterns for Large Language Models (RAG)

Intent: This document houses explicit calculations, cross-table constraints, financial balancing formulas, and operational pipeline logic rules. It is mapped out to prevent standard LLM hallucination errors regarding calculation fields, join conditions, and ambiguous filters.

Section 1: Order Valuation & Financial Calculation Logic

Rule 1.1: Order Line Item Totals (`order_items.line_total`)

The line total value for an individual ordered item must be computed deterministically using standard decimal calculations. It factors in quantity, baseline unit prices, and proportional percentage discounts.

$$line\_total = quantity \times unit\_price \times (1 - (discount\_percent / 100))$$

SQL Reference Standard:

```
SELECT item_id,
       (quantity * unit_price * (1.0 - (discount_percent / 100.0))) AS calculated_line_total
FROM order_items;
```

> **LLM Guardrail:** Never aggregate raw `unit_price` directly from the `products` table for historical order values. Always pull `unit_price` directly from `order_items`, as it represents the historical locked-in price at time of purchase.

Rule 1.2: Order Header Aggregation (`sales_orders` Totals)

The gross subtotal, applied tax amounts, and complete final financial totals are linked in a strict hierarchical constraint sequence across the order tables:

- **Subtotal:** Sum of all corresponding line totals. $subtotal = \sum(line\_total)$
- **Tax Amount:** Calculated on the subtotal using the default or product-specific rate constraints. $tax\_amount = subtotal \times tax\_rate$
- **Total Amount:** Net sum of subtotal and tax values. $total\_amount = subtotal + tax\_amount$

Deterministic Verification SQL:

```
SELECT o.order_id,
       COALESCE(SUM(i.line_total), 0) AS aggregated_subtotal,
       COALESCE(SUM(i.line_total), 0) + o.tax_amount AS calculated_total
FROM sales_orders o
JOIN order_items i ON o.order_id = i.order_id
GROUP BY o.order_id, o.tax_amount;
```

Section 2: Inventory Control & Stock Auditing Logic

Rule 2.1: Available Stock Reconciliation

Physical inventory parameters must follow a strict preservation rule. Free sellable stock, allocated amounts, and overall physical balance metrics are bound by the following equation:

$$available\_stock = stock\_on\_hand - allocated\_stock$$

Reorder Trigger Logic: A replenishment sequence is flagged as required when the following conditional rule evaluates to true:

$$stock\_on\_hand \leq reorder\_level$$

```
-- Query to identify products requiring immediate reordering
SELECT product_id, warehouse_name, stock_on_hand, reorder_level
FROM inventory
WHERE stock_on_hand <= reorder_level;
```

Rule 2.2: Stock Movement Ledger vs. Current Inventory Status

The state variable `inventory.stock_on_hand` must exactly equal the historical net sum of all logged stock mutations within the system ledger for that product-warehouse boundary.

$$stock\_on\_hand = \sum(quantity \ WHERE \ movement\_type = 'INBOUND') - \sum(quantity \ WHERE \ movement\_type = 'OUTBOUND')$$

```
-- Reconciliation audit query
SELECT i.product_id, i.warehouse_name, i.stock_on_hand,
       COALESCE(SUM(CASE WHEN m.movement_type = 'INBOUND' THEN m.quantity
                         WHEN m.movement_type = 'OUTBOUND' THEN -m.quantity
                         ELSE m.quantity END), 0) AS ledger_calculated_stock
FROM inventory i
LEFT JOIN stock_movements m ON i.product_id = m.product_id AND i.warehouse_name =
m.warehouse_name
GROUP BY i.product_id, i.warehouse_name, i.stock_on_hand;
```

Section 3: Double-Entry Accounting & Ledger Integrity

Rule 3.1: The General Ledger Balancing Constraint

The accounting system mandates an absolute double-entry bookkeeping constraint. Across the entire `general_ledger` table, or when grouped under any specific individual unique transaction event, the sum total of all debits must strictly equal the sum total of all credits.

$$\sum(debit) = \sum(credit)$$

> **LLM Guardrail:** If a user requests a validation check on system health, use this inequality constraint. Any state where `SUM(debit) != SUM(credit)` means an active transaction out-of-balance error exists.

```
-- Check for broken ledger entries
SELECT reference_order_id, SUM(debit) as total_debit, SUM(credit) as total_credit
FROM general_ledger
GROUP BY reference_order_id
HAVING SUM(debit) != SUM(credit);
```

Section 4: HR, Payroll & Organizational Hierarchy Structure

Rule 4.1: Department Management and Organizational Tree Joins

The organizational hierarchy contains a cyclical relationship path. `departments.manager_id` loops back into `employees.employee_id`, while `employees.dept_id` links into `departments.dept_id`.

- **To find an employee's manager:** Join employee to department on `dept_id`, then join department to manager on `manager_id = employee_id`.
- **To list department head details:** Join `departments d JOIN employees e ON d.manager_id = e.employee_id`.

```
-- Find employee name along with their direct Department Manager
SELECT e.employee_id,
       e.first_name || ' ' || e.last_name AS employee_name,
       d.dept_name,
       m.first_name || ' ' || m.last_name AS manager_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
JOIN employees m ON d.manager_id = m.employee_id;
```

Rule 4.2: Customer Credit Risk Threshold Monitoring

Customers are restricted from executing new automated purchases if their ongoing financial commitments exceed their verified limits. An account is out of credit guidelines if:

$$outstanding\_balance > credit\_limit$$

```
-- Identify high risk accounts blocking further orders
SELECT customer_id, company_name, credit_limit, outstanding_balance
FROM customers
WHERE is_active = true AND outstanding_balance > credit_limit;
```